Qwen3.5-0.8B playground

Installed setup and learning guide

AI Pilot • 11 October 2026 • Local Linux CPU playground

This guide documents the installed playground. Its purpose is to understand model analysis tools through small, observable experiments. It does not train the model or give it permanent memory.

One model, two ways to use it

The model is Qwen3.5-0.8B throughout. Llama, Meta's model family, is not installed. llama.cpp is the C++ runtime used here; it can run Qwen models.

Everyday chat: llama.cpp and its built-in web UI. This is the primary runtime. It uses a Q4_K_M GGUF conversion of Qwen3.5-0.8B from Unsloth, with quantized weights. The model file is 532,517,120 bytes, about 533 MB.

Analysis workshop: Hugging Face Transformers, CPU PyTorch and JupyterLab. This uses the original BF16 checkpoint, about 1.75 GB of model weights. Transformers supplies the model implementation; PyTorch performs the tensor operations and exposes hooks; JupyterLab displays the interactive notebook. BertViz renders captured attention weights. Matplotlib plots layer activations. The project helpers connect those tools; they are not a new inference engine.

Quantization changes numerical values. The two runtimes use the same model family and size, but are not expected to produce identical logits or replies. Do not attribute differences solely to the programming language or runtime.

Start and stop

From a terminal in `/home/raxim/workspace/ai-pilot`:

```
./tools/start-chat.sh
```

Open http://127.0.0.1:8088 for the C++ chat. The server listens only on the local computer. Its default settings are two CPU threads, one parallel slot, a 2048-token context and thinking disabled. No GPU backend is enabled.

For internal analysis, stop the chat with Ctrl+C, then run:

```
./tools/start-playground.sh
```

Open the authenticated URL printed by Jupyter. It opens the analysis notebook on port 8890. Choose **Run → Run All Cells**, then **Analyze prompt**. Notebook initialization creates the controls; it does not load model weights yet.

Closing a browser tab does not stop either server. Use Ctrl+C in the launch terminal, Jupyter's **File → Shut Down**, or this project-owned stop command:

```
python3 tools/stop-playground.py
```

Nothing starts when the computer boots or when you log in. Setup installs no systemd service, cron task, login hook, desktop autostart item or Docker restart policy. Servers run only after explicit launch. Both runtimes operate locally after installation; BertViz's browser dependencies are also local.

What the analysis screens show

Tokens. The exact token IDs and pieces fed into the model. Words can split into multiple tokens, particularly across languages. Raw text shows completion behavior. Chat-template mode adds the role delimiters and non-thinking assistant prefix used for an instruction-following conversation.

Next token. The ten highest probabilities after applying softmax over the entire vocabulary to the final position's logits. These probabilities describe the next token, not whether a statement is true. Only the ten largest values are displayed, so their displayed probabilities need not sum to one.

Attention map. Choose one full-attention layer and one of its eight query heads. Rows represent query tokens, columns the earlier/current tokens they attend to. The future-token triangle is causally masked. Brighter cells mean larger attention weights for this particular prompt and head.

BertViz. An interactive connection view of those same attention weights. Select layers and heads, then inspect token connections. BertViz's compact layer numbers 0–5 map to actual Qwen layer indices 3, 7, 11, 15, 19 and 23. These are zero-based indices. The specialized BertViz neuron view is not enabled for Qwen3.5 in this playground.

All 24 layers. A heatmap of each decoder block's output RMS over its 1024 hidden dimensions, for every token position. Hooks capture outputs after the token mixer, feed-forward computation and residual additions. This covers both DeltaNet and full-attention blocks. Brightness is activation magnitude, not a measure of importance, confidence or a named semantic concept.

Why only six attention layers?

Qwen3.5-0.8B has 24 decoder layers: three Gated DeltaNet blocks followed by one full-attention block, repeated six times. It is a dense hybrid language model, not a tiny mixture-of-experts model.

DeltaNet uses a different state-based token-mixing computation. It does not provide the same softmax token-to-token matrix as full attention. The current workshop measures its decoder outputs; it does not display a decoded recurrent memory state, nor manufacture an attention matrix for those 18 blocks.

The downloaded original checkpoint also contains a vision encoder. This first playground exercises the text path only. The C++ setup does not download a vision projector. Image/video analysis is not part of the verified setup.

Interpretation limits

Token flow adds a Hungarian, step-through representation graph to the analysis tabs. It shows the first 16 measured coordinates of each 1024-wide vector, decoder stages, matrix shapes and the final next-token distribution. See `docs/TOKEN_REPRESENTATION.md`; the exported HTML example works offline without a model. Playback replays recorded values, not real-time inference.

Attention weights and large activations are observations, not complete causal explanations. A generated reasoning narrative is text, not a dump of internal computation. The experiments do not identify concepts inside neurons or prove how the model reasons. A controlled intervention provides stronger evidence about one component's effect on one prompt, with limited generalization.

First experiments

1. Change one fact. Analyze `The capital of Hungary is` in raw-text mode. Inspect Tokens and Next token. Replace Hungary with France and compare. Write down the top token, its probability and the token count. You are examining a completion, not asking a chat-formatted question.

2. Inspect the chat wrapper. Enable Chat template and analyze the same text. Observe the added control tokens and changed next-token distribution. Generate reply always uses chat mode with thinking disabled, regardless of the analysis checkbox. It returns at most the selected output-token limit.

3. Compare attention heads. Use the same prompt and select layers 3 and 23, then several heads. Note which positions they attend to. Keep the prompt unchanged so the comparison has a clear meaning. Different heads need not have simple human-readable roles.

4. Compare decoder activations. Inspect All 24 layers, then replace one word and repeat. Compare the scale as well as the colors: each plot has its own color range. This is a compact RMS summary, not every individual neuron.

5. Perform a reversible intervention. Select MLP layer 0 and press Compare MLP ablation. The helper runs the same input once normally, then with that layer's feed-forward output replaced by zero. It compares the full next-token distributions and reports total variation, from 0 to 1. Model weights are unchanged. The hook is removed in a finally block, including on errors.

If the top token does not change, that is still a meaningful observation: other probabilities may have moved. This intervention only tests the selected MLP output for the current prompt. It does not isolate the DeltaNet mixer, establish a universal function for the layer or demonstrate learning.

Resource controls

The analysis input is limited to 96 tokens, including chat delimiters. Longer input is rejected before model loading rather than truncated silently. Generation is limited to 64 new tokens and can therefore end mid-answer. Each inference uses two CPU threads and no gradients. Controls disable while an action runs. The loader asks for at least 3 GiB of currently available RAM.

The model stays loaded between experiments for convenience. Unload model releases its reference; Python's allocator may retain memory. **Kernel → Shut Down Kernel** reliably ends the kernel process. Use one loaded model runtime at a time when conserving RAM. No analysis runs in the background on a timer.

The CPU analysis path reports that optional fast DeltaNet kernels are absent. This is expected: it uses PyTorch's reference operations. The optimized C++ chat is the primary runtime; the reference path is for bounded inspection.

The full installation uses roughly 3.8 GB before installer caches: about 46 MB for the C++ package, 533 MB for GGUF, 1.75 GB for original weights and 1.5 GB for the Python environment. Additional tokenizer files and experiment outputs take some space. These are disk sizes, not runtime RAM requirements.

Verification on this computer

Host: Intel Core i5-1335U, approximately 16 GB physical RAM, CPU-only inference. The unrelated application deployment was not restarted or modified for this setup.

The analysis check used `The capital of Hungary is` (five tokens). It captured all 24 decoder outputs and the six expected full-attention layers, each with eight query heads. Checks covered finite values, attention row normalization, the causal future-token mask, next-token probabilities, the input limit, intervention-hook cleanup and real chat generation. A standalone BertViz HTML was rendered in a browser with a visible SVG and layer selector.

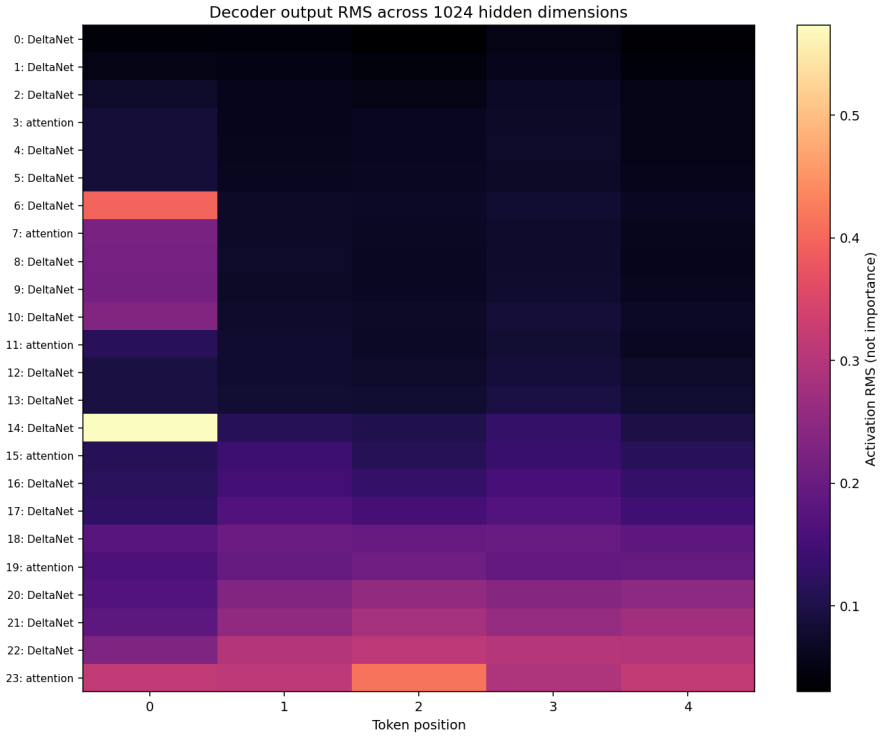
Jupyter's actual controls were also exercised in a browser: Run All Cells, Analyze prompt, the activation tab, the embedded BertViz iframe and Unload model. Both server processes were stopped with the project command and restarted explicitly. No reboot test was needed or performed.

The initial analysis forward pass took approximately 0.75 seconds, excluding model loading. A short chat prompt produced `Budapest.` in about 3.52 seconds. The complete analysis-check process peaked near 2046 MiB resident memory. Zeroing MLP layer 0 changed the next-token distribution by approximately 0.095 total variation on the example prompt.

The C++ server returned healthy status and generated `Budapest` through its chat-completions endpoint. The request also returned token log-probabilities. That short request took approximately 0.55 seconds, and the server used about 847 MiB resident memory after it. Its built-in browser UI displayed the correct Qwen3.5 0.8B model.

These are **smoke-check observations, not a matched performance benchmark**. The runtimes used different weight precision and may differ in templates, token counts and cache behavior. These timings do not isolate a C++ versus Python speed effect or predict performance on longer prompts.

Example captured activation map



Actual 24-layer decoder output RMS for the five-token example

Reproduction and files

`tools/setup-llama.sh` downloads the official llama.cpp Linux x86-64 binary bundle and the GGUF. It verifies both SHA-256 hashes before use. The runtime is build b11429, commit d81235049, distributed by the v0.6.0 release.

`tools/setup.sh` creates the repository-local `.venv`, installs CPU-only PyTorch, installs the analysis dependencies, registers a kernel inside that environment and downloads the pinned original checkpoint. It does not modify the system Python or install a global Jupyter kernel. `requirements.lock.txt` records all resolved Python package versions.

Key versions: Transformers 5.19.0; PyTorch 2.8.0+cpu; JupyterLab 4.4.10; BertViz 1.4.1. The checkpoint revision is `2fc06364715b967f1860aea9cf38778875588b17`. The GGUF repository revision is `6ab461498e2023f6e3c1baea90a8f0fe38ab64d0`.

The original weights live in `models/Qwen3.5-0.8B/`. The GGUF lives in `models/Qwen3.5-0.8B-Q4_K_M.gguf`. Neither is committed to Git. Experiment CSVs, images, JSON evidence and browser screenshots are in ignored `artifacts/`. Runtime logs, Jupyter credentials and server state are in ignored `.runtime/`.

To rerun the focused analysis check after shutting down loaded model runtimes:

```
.venv/bin/python tools/check-analysis.py
```

To regenerate this PDF from this Markdown and the captured activation image:

```
.venv/bin/python tools/build-guide.py
```

Recreating the installation requires network access. Normal inference uses local files only. Jupyter keeps its normal token authentication; use the URL printed by the launcher and do not publish the `.runtime/` directory.

Primary references

Qwen3.5-0.8B model card

Transformers Qwen3.5 implementation and API

llama.cpp official repository

Official C++ binary release used here

Unsloth Qwen3.5-0.8B GGUF conversion

BertViz documentation

JupyterLab user guide